

Experiment No: 02

Experiment Name / Title: Network Traffic Analysis and TLS Certificate Inspection Using Wireshark

Aim or Objectives

1. To understand how network traffic is captured and how packets are structured into protocol layers.
2. To use Wireshark to capture live traffic and to identify common protocols such as ICMP, DNS, TCP, HTTP, and TLS.
3. To apply display filters in order to isolate specific traffic within a capture.
4. To observe the TLS handshake and identify the messages exchanged before encryption begins.
5. To inspect the fields of a digital certificate and explain how the chain of trust is verified.

Apparatus Required

- A computer with Kali Linux installed, or a Linux virtual machine
- Wireshark packet analyser
- A web browser such as Firefox
- Internet access or a local web server within the lab environment
- Network access within an isolated lab environment

Packet Capture and Network Traffic Analysis

All data sent across a network travels in small units called packets. A packet analyser records copies of the packets that pass through a network interface and displays their contents in a readable form. Wireshark is the most widely used packet analyser and is available free of charge.

In the previous experiment, Nmap was used to scan a target. Nmap is an active tool, since it sends probes and prints a summary of the replies. Wireshark is a passive tool, since it sends nothing of its own and only records traffic that is already present. Nmap answers the question of which services exist on a host, while Wireshark answers the question of what those services are actually saying.

The Wireshark Interface

The Wireshark window is divided into three panes. The packet list pane at the top shows one line for each captured packet. The packet details pane in the middle shows the protocol layers of the selected packet and can be expanded. The packet bytes pane at the bottom shows the raw data of the packet.

The packet list pane has the columns No., Time, Source, Destination, Protocol, Length, and Info. The Protocol column shows the highest layer that Wireshark was able to identify, not the only protocol present in the packet.

Protocol Layers and Encapsulation

Each packet is built from layers. The message produced by the application protocol is wrapped by the transport layer, that result is wrapped by the network layer, and that result is wrapped again by the link layer before the packet is sent. Expanding a packet in the details pane shows these layers from the outside inwards, and the structure is the same for every packet examined in this experiment.

Layer	Information it carries
Frame	Details recorded by Wireshark, such as capture time and frame length
Ethernet II	Source and destination MAC addresses, used for delivery on the local network
Internet Protocol	Source and destination IP addresses, used for delivery between networks
TCP or UDP	Source and destination port numbers, used to reach the correct service
Application protocol	The actual message, for example DNS, HTTP, or TLS

Capture Filters and Display Filters

Wireshark supports two different kinds of filter and they must not be confused. A capture filter is set before the capture begins and permanently discards any traffic that does not match, using a syntax such as port 443. A display filter is applied after the capture and only hides packets from view, so nothing is lost and the filter can be changed at any time, using a syntax such as tcp.port == 443. All filtering in this experiment uses display filters.

Common Protocols Observed in a Capture

Protocol	Purpose	Port
ARP	Finds the MAC address that belongs to an IP address on the local network	None
ICMP	Tests whether a host is reachable, used by the ping command	None
DNS	Translates a domain name into an IP address	53
TCP	Provides a reliable connection oriented transport service	Varies
UDP	Provides a fast connectionless transport service	Varies
HTTP	Carries web traffic without encryption	80
TLS	Encrypts traffic carried over TCP, used by HTTPS	443

The TCP Three Way Handshake in a Capture

Before any data is exchanged, two devices using TCP complete a three way handshake of SYN, then SYN-ACK, then ACK. In Wireshark these appear as three separate packets at the start of every TCP connection, and the Info column names the flags directly.

This connects to the previous experiment. A TCP SYN scan in Nmap sends the first SYN and reads the reply. An open port replies with SYN-ACK, a closed port replies with RST, and a filtered port sends no reply at all. Nmap prints only the conclusion, while Wireshark shows the packets that led to it.

HTTP and the Problem of Plaintext

HTTP carries data with no encryption of any kind. If a user submits a login form over HTTP, the username and password travel across the network as readable text, and anyone capturing that traffic can recover them. This weakness is the reason encrypted web traffic was introduced, and it is demonstrated in this experiment using a public test website and false credentials only.

TLS (Transport Layer Security)

TLS is the protocol that provides encryption for data carried over TCP. HTTPS is simply HTTP carried inside TLS, normally on port 443. Before any encrypted data is exchanged, the two sides complete a handshake in which they agree on the rules of the connection and the server proves its identity.

Message	Purpose
Client Hello	The client lists the TLS versions and cipher suites it supports and states the hostname it wants to reach in the Server Name Indication field.
Server Hello	The server selects one TLS version and one cipher suite from the list offered by the client.
Certificate	The server presents its digital certificate so that the client can verify its identity.
Key Exchange	Both sides establish a shared secret key without sending that key across the network.
Change Cipher Spec	Both sides confirm that encryption begins from this point onward.
Application Data	All further traffic is encrypted and appears in Wireshark only as Application Data.

The version of TLS in use affects what can be seen. In TLS 1.2 the Certificated message is sent before encryption starts and is fully visible in Wireshark. In TLS 1.3 the certificate is encrypted immediately after the Server Hello and is no longer visible, which is a deliberate improvement to privacy rather than a fault in the capture. A server that still offers TLS 1.2 is therefore used for certificate inspection in this experiment.

Encryption protects the content of a connection, but not the fact that the connection took place. The DNS query and the Server Name Indication field in the Client Hello both reveal the name of the site being visited, and both are sent without encryption.

Digital Certificates and the Chain of Trust

A digital certificate is an electronic document that binds a public key to a named owner and is signed by a Certificate Authority, abbreviated as CA. The signature allows a client to confirm that the certificate was issued by a body it already trusts.

Certificate Field	Meaning
Subject, CN	The name the certificate was issued to
Issuer, CN and O	The Certificate Authority that signed the certificate
Validity, notBefore and notAfter	The period during which the certificate is valid
Serial Number	A unique number assigned to the certificate by the CA
Subject Public Key Info	The public key algorithm and key size, for example RSA 2048
signatureAlgorithm	The algorithm used by the CA to sign, for example sha256WithRSA
Subject Alternative Name	The list of hostnames that the certificate is valid for

A server normally sends more than one certificate. Its own certificate is signed by an intermediate CA, the intermediate certificate is signed by a root CA, and the root certificate is already stored in the trust store of the operating system or browser. This sequence is called the chain of trust, and if any link in it fails the browser displays a warning.

Common Certificate Errors

Error	Field responsible
Certificate has expired	The notAfter date has already passed
Name does not match	Subject Alternative Name does not include the hostname requested
Self signed certificate	Issuer is the same as Subject, so no CA has verified it
Untrusted root	The root CA is not present in the trust store of the system

The Cybersecurity Testing Lifecycle and the Role of Wireshark

Wireshark supports every phase of the cybersecurity testing process, because it records the evidence of what actually travelled across the network.

Phase	Role of Wireshark
1. Reconnaissance	Passively records traffic to learn which hosts, protocols, and services are in use.
2. Scanning and Enumeration	Shows the probe and reply packets that produced a scan result.
3. Vulnerability Assessment	Reveals plaintext credentials, weak protocols, outdated TLS versions, and invalid certificates.
4. Exploitation	Wireshark does not attack a system, but it confirms whether traffic sent during a test reached the target.
5. Post-Exploitation	Detects unusual outbound connections from a host that has been compromised.
6. Reporting	Saved capture files are used as evidence in the final report.

Why Wireshark Matters

Wireshark is free, open source, and runs on Windows, Linux, and mac OS. Network administrators use it to troubleshoot faults, security analysts use it to investigate incidents, and penetration testers use it to confirm how a target responds. It is also used by attackers to collect information from traffic they can observe. Reading a capture requires an understanding of how protocols work, so learning Wireshark strengthens networking knowledge at the same time. For these reasons it is one of the first tools a cybersecurity student learns after a scanner such as Nmap.

Identifying the Capture Interface

A capture is always taken on one network interface. The active interface must be identified before the capture begins, and in a virtual machine using network address translation it is usually eth0.

Syntax: *ip addr*

Filtering by Protocol

A protocol name entered on its own in the display filter bar shows only the packets of that protocol. If the filter bar turns red, the filter contains a syntax error.

Syntax: *icmp*

Syntax: *dns*

Syntax: *http*

Syntax: *tls*

Filtering by Host and Port

A capture can also be narrowed to a single address or a single port, which is useful when many connections are active at once.

Syntax: *ip.addr == 8.8.8.8*

Syntax: *tcp.port == 443*

Filtering TLS Handshake Messages

Each handshake message has a numeric type, so a single message of the handshake can be isolated directly.

Syntax: *tls.handshake*

Syntax: *tls.handshake.type == 1 (Client Hello)*

Syntax: *tls.handshake.type == 2 (Server Hello)*

Syntax: *tls.handshake.type == 11 (Certificate)*

Following a Stream

Following a stream reassembles all packets of one conversation and displays them as a single readable exchange. Right click any packet of the connection, choose Follow, then choose TCP Stream or HTTP Stream.

Saving a Capture

A capture is saved for later analysis or as evidence using File, then Save As, in the pcapng format.

Syntax: *capture.pcapng*

Procedure

Follow these steps in order. Capture only the traffic that you generate on your own machine. Capturing the traffic of other people without written authorisation is an offence under computer misuse law and is against university policy. Use the public test websites named below and never enter real credentials.

```

# Step 1: Identify the active network interface
ip addr

# Step 2: Open Wireshark, select the active interface, and start the capture

# Step 3: Generate ICMP traffic, then apply the display filter
ping -c 4 8.8.8.8
Display filter: icmp

# Step 4: Generate DNS traffic, then apply the display filter
nslookup example.com
Display filter: dns

# Step 5: Observe unencrypted web traffic
# Visit http://testphp.vulnweb.com and submit false credentials
Display filter: http
# Right click the POST packet, then Follow, then HTTP Stream

# Step 6: Observe the TLS handshake
# Visit https://tls-v1-2.badssl.com:1012
Display filter: tls.handshake

# Step 7: Inspect the Client Hello and the Server Hello
Display filter: tls.handshake.type == 1
Display filter: tls.handshake.type == 2

# Step 8: Inspect the server certificate and record its fields
Display filter: tls.handshake.type == 11

# Step 9: Record the browser warning for an invalid certificate
# Visit https://expired.badssl.com

# Step 10: Stop the capture and save it
# File, then Save As, then capture.pcapng

```

Observation Table:

Target Website	Display Filter used	Filter Information	Output

Conclusion

In your own words, summarize the protocols you identified and the messages you observed in the TLS handshake. State the issuer, subject, and expiry date of the certificate you inspected, and explain what an observer can and cannot learn about an HTTPS connection.